

PROJECT REPORT

Design, Synthesis and Verification of an Asynchronous FIFO for Clock Domain Crossing (CDC)

Gray-Code Pointer Based Dual-Clock FIFO Architecture

Name: Manish Ranjan

Roll Number: 24M1176

Institute: Indian Institute of Technology Bombay

Tools Used: Xilinx Vivado (Simulation), Cadence Genus (Synthesis)

1. Objective

The goal of this project is to design and verify an asynchronous FIFO (First-In-First-Out) buffer that can safely move data between two independent, unrelated clock domains — a write clock (`wr_clk`) and a read clock (`rd_clk`) — without data corruption or metastability, and to confirm correct behaviour through simulation and Cadence Genus synthesis.

2. Why an Asynchronous FIFO Is Needed

Whenever two parts of a chip run on different, unrelated clocks (for example, a fast processing core and a slower peripheral interface), any signal that crosses from one clock domain to the other risks metastability — a flip-flop can be sampled while it is still changing value, producing an unpredictable output. An asynchronous FIFO is the standard, industry-proven way to move a stream of data safely across this boundary.

A good async FIFO must guarantee four things, in simple terms:

- No metastability — signals crossing domains are properly synchronized.
- No pointer corruption — read/write address pointers stay valid even mid-transition.
- Reliable FULL / EMPTY detection — the FIFO never overflows or underflows.
- Simple, synthesizable logic — fully implementable in standard-cell silicon.

3. Design Theory

3.1 Why Gray-Code Pointers?

A normal binary counter can change several bits at once when it increments. For example:

```
0111 -> 1000    // all 4 bits change together
```

If this value is sampled by a flip-flop in a different, unrelated clock domain exactly while it is transitioning, the sampled value can be completely wrong (not just one bit off) — this is called pointer corruption.

Gray code solves this because only one bit changes for every increment:

```
0110 -> 0111 -> 0101 -> 0100 -> 1100 -> ...
```

So even if the receiving domain samples the pointer at the exact moment it is changing, the worst case is a 1-bit uncertainty — never a completely wrong value. This is what makes Gray-code pointers safe to pass through a synchronizer.

3.2 Pointer Width — the Extra MSB

If the FIFO has $\text{DEPTH} = 2^{\text{ASIZE}}$ locations, each pointer is made ONE bit wider than the address, i.e. $\text{width} = \text{ASIZE} + 1$:

- The lower ASIZE bits select the actual memory address.
- The extra MSB (top bit) is used purely to tell whether the pointer has wrapped around ("lapped") the memory once.

This extra bit is the key trick that lets FULL and EMPTY be told apart even though, in both cases, the write and read addresses point at the same physical memory location.

3.3 Two-Flop Synchronizers (Crossing Clock Domains)

Each Gray-coded pointer is passed into the opposite clock domain through a chain of two flip-flops (a standard double-flop synchronizer). Only the second stage's output is actually used by the logic — the first stage exists purely to let any metastability resolve before the value is used:

```
// Read pointer -> write clock domain
wrptr_s1 <= rd_gray;
wrptr_s2 <= wrptr_s1;

// Write pointer -> read clock domain
rwptr_s1 <= wr_gray;
rwptr_s2 <= rwptr_s1;
```

This is why the constraint `set_clock_groups -asynchronous` is applied between `wr_clk` and `rd_clk` during synthesis: it tells the STA tool these two clocks have no fixed timing relationship, so it should not try to check setup/hold across them, and instead rely on this synchronizer structure for safe crossing.

3.4 Next-Pointer Logic

The FULL and EMPTY flags are computed one cycle early, from the "next" pointer value rather than the current one. This makes the flags update immediately on the same clock edge that the write/read actually happens, instead of lagging by a cycle:

```
wbin_next = wr_bin + (wr_en & ~full);
wgnext    = bin2gray(wbin_next);

rbin_next = rd_bin + (rd_en & ~empty);
rgnext    = bin2gray(rbin_next);
```

3.5 FULL Condition

FULL means the writer has gone exactly one full lap ahead of the reader and caught up to the same physical memory slot. In binary-pointer terms (easiest to understand first), this is checked as:

- The extra MSBs of the two pointers are different (writer has wrapped once more than reader), AND
- The lower address bits of the two pointers are equal (same physical slot).

Worked example (3-bit address, so pointer width = 4 bits, depth = 8): if `wr_ptr = 1000` and `rd_ptr = 0000`, the MSBs differ (1 vs 0) but the lower 3 bits are equal (000 = 000) — so FULL is asserted. If instead `rd_ptr = 0010`, the lower bits (000 vs 010) do NOT match, so the FIFO is not full — there are simply unread entries still sitting between the two pointers.

In the actual hardware, this same comparison is done directly on the synchronized Gray-coded pointers (not raw binary), which is why the RTL inverts the top two bits of one pointer before comparing:

```
wfull_reg <= (wgnext == {~wrptr_sync[P-1:P-2], wrptr_sync[P-3:0]});
```

Gray code doesn't increment/compare the same way binary does, so instead of subtracting pointers, the FULL check flips the top two bits of the synchronized write pointer and compares directly — this is the standard, industry-accepted way to detect "one full lap ahead" purely in Gray code.

3.6 EMPTY Condition

EMPTY is the simpler case — it just means the next read pointer would equal the already-synchronized write pointer, i.e. the reader has completely caught up to the writer and there is nothing left to read:

```
rempty_reg <= (rgnext == rwptr_sync);
```

3.7 Memory Read/Write

```
// Write (write-clock domain)
if (wr_en && !full)
    mem[waddr] <= wr_data;

// Read (read-clock domain)
rd_data <= mem[raddr];
```

Writes only happen when `wr_en` is asserted AND the FIFO is not full — this is what protects against overflow. Similarly, the testbench only pulses `rd_en` while empty is low, protecting against underflow.

3.8 Why FIFO Depth Must Be a Power of 2

Gray code only wraps around cleanly (one bit changing per step, all the way around and back to the start) when the count sequence length is a power of 2. A non-power-of-2 depth would break the one-bit-transition property right at the wrap point, which would defeat the whole purpose of using Gray code for safe CDC.

4. Testbench Description

The testbench (`tb_async_fifo_simple`) verifies the FIFO end-to-end using two independent, free-running clocks and a directed test sequence.

4.1 Clock Generation

- Write clock (`wr_clk`): 100 MHz (10 ns period)
- Read clock (`rd_clk`): ~71 MHz (14 ns period)

Because these two periods are unrelated to each other, the two domains genuinely operate asynchronously — exactly the condition the FIFO is designed to handle.

4.2 Reset Sequencing

In post-implementation timing simulation, Vivado holds an internal Global Set/Reset (GSR) active for about the first 100 ns. The testbench waits 200 ns before releasing `wr_rst_n` and `rd_rst_n`, so the design's real reset logic — not the simulator's GSR — is what actually initializes the flip-flops.

4.3 Test 1 — Fill the FIFO to FULL

`wr_en` is held high and the testbench keeps writing incrementing data (0x00, 0x01, 0x02 ...) on every `wr_clk` edge until the full flag goes high. It then attempts one extra write (0xAA) after FULL is asserted. Because the

RTL only writes when (`wr_en && !full`), this extra write is correctly ignored — this is the overflow-protection check.

4.4 Test 2 — Drain the FIFO to EMPTY

`rd_en` is held high and the testbench keeps reading on every `rd_clk` edge until the empty flag goes high, confirming the FIFO drains completely and correctly reports EMPTY once all written data has been read back out, in the same order it was written (FIFO ordering preserved across the clock-domain crossing).

5. Simulation Results (Waveforms)

Post-implementation timing simulation was run in Vivado, using the actual placed-and-routed netlist with back-annotated delays, to confirm the FULL and EMPTY conditions behave correctly under real timing.

5.1 FULL Condition

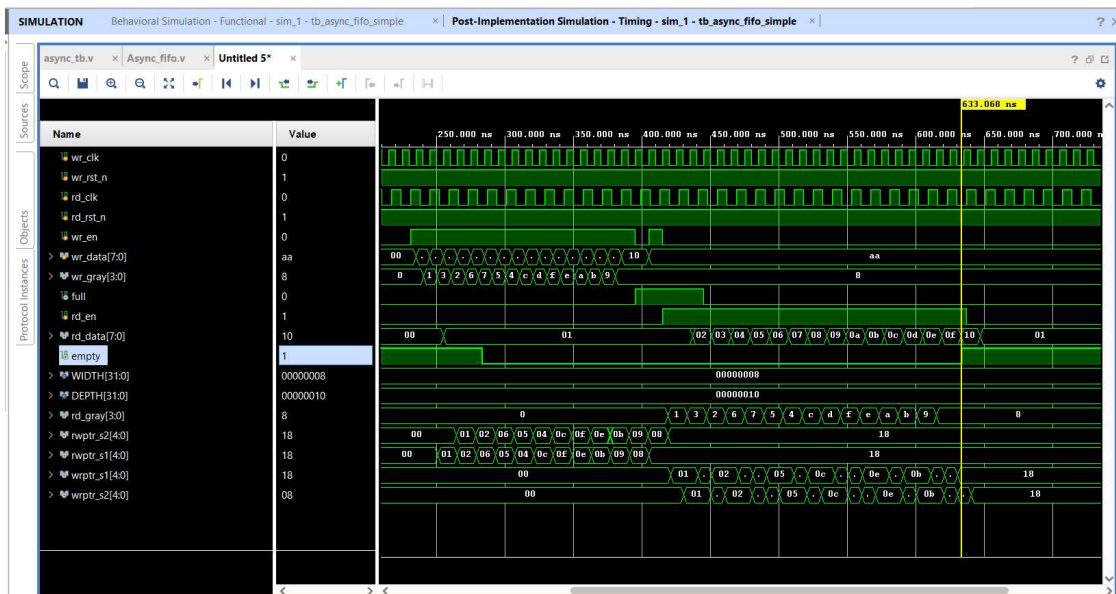


Figure 1: Waveform showing `wr_gray` reaching 8 while `rd_gray` is still at 0 — the write pointer has lapped the read pointer, so `full = 1`. Writes stop being accepted at this point (`wr_data` holds at 0xaa, the rejected extra write).

Here, `wr_gray` has climbed through the full 4-bit Gray sequence and reached 8 (1000) while `rd_gray` is only just starting to increment from 0. The synchronized write pointer (`wrptr_s2`) matches this condition against the read pointer, and `full` asserts — matching the Section 3.5 theory exactly.

5.2 EMPTY Condition

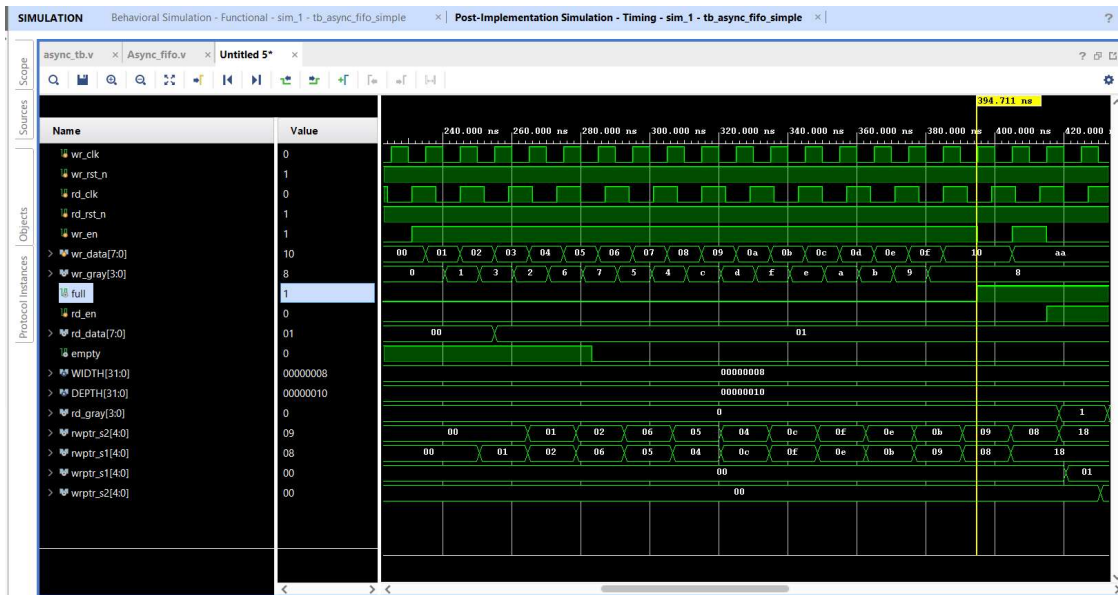


Figure 2: Waveform showing the FIFO being drained. Once rd_gray catches up to the synchronized write pointer, empty = 1, and rd_data holds its last valid value (0x01).

As rd_en pulls data out on every rd_clk edge, rd_gray steadily climbs. Once the read pointer's next value equals the synchronized write pointer (rwptr_s2), empty asserts and reading stops — again matching the Section 3.6 theory.

6. Synthesis Results (Cadence Genus)

The design was synthesized on Cadence Genus targeting the standard-cell library, to check area, timing, and power of the async FIFO in real silicon terms.

6.1 Area

Module	Cell Count	Total Area (μm^2)
fifo_async	502	17,219.76

502 standard cells were used, giving a compact total cell area of about 17,220 μm^2 — reasonable for a small utility block like a FIFO.

6.2 Timing

The critical path checked was the setup path into wfull_reg (the FULL flag register) on the wr_clk domain:

Parameter	Value
Clock period (wr_clk)	10,000 ps (100 MHz)
Data path delay	3,616 ps
Setup slack	6,170 ps (MET)

The path comfortably meets timing with over 6 ns of positive slack at 100 MHz, meaning the FULL-flag logic (which chains through several gates computing the Gray-code comparison) is not a bottleneck at this clock speed.

6.3 Power

Category	Leakage (W)	Internal (W)	Switching (W)	Total (W)	% of Total
Register	1.41e-07	1.09e-03	3.10e-05	1.12e-03	79.4%
Logic	7.79e-08	1.30e-04	6.12e-05	1.92e-04	13.6%
Clock network	0	0	9.93e-05	9.93e-05	7.0%
Total	2.19e-07	1.22e-03	1.92e-04	1.41e-03	100%

Total power is about 1.41 mW, dominated by the registers ($\approx 79\%$) — expected, since the pointers, synchronizer flops, and FULL/EMPTY flags are all sequential logic that toggles on every clock edge.

7. Conclusion

- Async FIFO = safe way to move data between two unrelated clocks.
- Gray code -> only 1 bit changes per step -> safe to sample mid-transition.
- Pointers are (ASIZE+1) bits wide -> extra MSB distinguishes FULL from EMPTY when addresses match.
- 2-flip-flop synchronizers pass each pointer into the opposite clock domain; only the 2nd stage is used.
- FULL = MSBs differ + lower bits equal (one full lap ahead, same slot).
- EMPTY = next read pointer equals synchronized write pointer (fully caught up).
- set_clock_groups -asynchronous tells STA there's no fixed relation between wr_clk/rd_clk, so it won't falsely flag cross-domain paths — synchronizers, not STA, guarantee CDC safety.
- Depth must be a power of 2 for Gray code to wrap correctly.
- Verified in Vivado (behavioral + post-implementation timing simulation) and synthesized in Cadence Genus — timing met with 6.17 ns slack, ~ 1.41 mW total power.